

CLOUD-NATIVE MICROSERVICES PLATFORM WITH CI/CD AND OBSERVABILITY

PROJECT SYNOPSIS
OF INDUSTRIAL TRAINING (TR-104)

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE ENGINEERING

SUBMITTED BY
Gurpiyar Singh
URN: 2302529



GURU NANAK DEV ENGINEERING COLLEGE,
LUDHIANA

Candidate and Training Details

1. Name of Student: Gurpiyar Singh

2. PTU (University) Roll No.: 2302529

3. Class Roll No.: 2315072

4. Present Official Address: AppSmartz, 2nd Floor, Quark City, Sector 75, Mohali, Punjab, India 160059 — E-mail: gurpiyar656@gmail.com, Telephone: +91-9803822976

5. Branch: Computer Science Engineering

6. Batch: 2023–2027

7. Proposed Topic: Cloud-Native Microservices Platform with CI/CD and Observability

Training Organization: AppSmartz

Duration of Training: 29-06-2026 to 31-12-2026

Organization Mentor: Mr. Harshdeep Singla, Tech Lead

College Guide: Prof. Shailja

Live Deployment: <https://learning.run.place>

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	System Architecture Overview	3
1.3	Automated Build and Delivery Pipeline	3
1.4	Secure Hosting and Observability	4
1.5	Glossary of Key Terms	4
2	Feasibility Study	5
2.1	Technical Feasibility	5
2.2	Economic Feasibility	5
2.3	Operational Feasibility	5
2.4	Need and Significance	5
3	Methodology / Planning of Work	7
3.1	Development Stages	7
3.2	System Architecture Diagram	8
4	Facilities Required for Proposed Work	9
4.1	Software	9
4.2	Hardware	9
	Bibliography	10

1 Introduction

1.1 Background and Motivation

My industrial training at AppSmartz was spent with the backend/DevOps team, observing at close quarters the mechanics by which software transitions from a developer's workstation to a resilient, production-grade system: decomposition into loosely-coupled services, automated integration and delivery, traffic arbitration through a reverse proxy, and continuous instrumentation of the running system's health. This synopsis proposes a project that condenses that entire lifecycle into a scale tractable for a training period, while retaining the fidelity of the underlying paradigm – architecture, containerization, CI/CD, secure public exposure, and observability are all genuinely implemented, not merely narrated.

1.2 System Architecture Overview

The platform itself is an e-commerce-style system decomposed into twelve independent, single-responsibility Node.js/Express microservices – user, product, order, cart, inventory, payment, notification, review, auth, shipping, search and analytics – each encapsulating its own data and its own Dockerfile. None is exposed externally; every inbound request is arbitrated by a single **API Gateway**, which proxies each `/api/...` route to its corresponding service and presents one coherent surface to the outside world. A **frontend dashboard** renders this surface as a live health-check grid, an animated request console, and reactive data tables, while the API contract is formalised as an **OpenAPI 3** specification and exposed interactively through **Swagger UI** at `/api-docs`.

1.3 Automated Build and Delivery Pipeline

Delivery is fully automated. Every service is containerized with **Docker**; **Docker Compose** orchestrates the ensemble, with a development variant that builds images from source and a production variant that pulls immutable, pre-built artifacts. Bridging the two is a **Jenkins** pipeline that checks out the repository, runs the codebase through a **SonarQube** static-analysis quality gate to catch maintainability and security issues before they compound, builds and tags an image

per service, pushes those images to Docker Hub, and finally deploys the production stack to the remote host over SSH – a declarative sequence that removes the human from the critical path between a commit and a release.

1.4 Secure Hosting and Observability

Two capabilities are being layered onto this foundation specifically because of practices observed at AppSmartz that had not previously been attempted: a **secured public domain** and a **dedicated observability stack**. The platform is reachable at <https://learning.run.place> through an **Nginx** reverse proxy terminating **TLS/SSL** via a Let's Encrypt/Certbot certificate, so the system is a genuine HTTPS deployment rather than a `localhost` artefact. Alongside it, **Prometheus** scrapes request-rate, latency and error telemetry from every service, **Promtail** ships container logs into **Grafana Loki** for centralised aggregation, and **Grafana** dashboards unify both signals so the cluster's behaviour is legible from a single pane rather than reconstructed service by service. Together, static analysis, encrypted ingress and full-stack telemetry are what separate a demo from a platform – and, should the mesh outgrow a single Compose deployment, the same container images translate directly onto an orchestrator such as **Kubernetes** without modification.

1.5 Glossary of Key Terms

A brief glossary for terms used throughout: a **microservice** is a small, independently deployable unit owning one business capability; an **API gateway** unifies many such services behind one address; a **reverse proxy** accepts public traffic and forwards it internally while handling cross-cutting concerns such as TLS termination; a **CI/CD pipeline** automates the path from a code change to a running deployment; a **quality gate** (SonarQube, here) blocks a build that fails defined code-health thresholds; and **observability** denotes the ability to reason about a running system from its metrics, logs and traces rather than by inference.

2 Feasibility Study

2.1 Technical Feasibility

Every tool used in the project – Node.js, Express, Docker, Docker Compose, Jenkins, SonarQube, Nginx, Prometheus, Loki and Grafana – is open source, widely documented, and already partly proven inside this project: the fourteen services, the API gateway, the Jenkins pipeline and the Swagger documentation are already built and run correctly in Docker Compose. What remains (the Nginx/SSL front door and the Prometheus–Loki–Grafana stack) uses the same container-based approach as the rest of the project, so no new deployment model has to be learned, only new containers added to the same compose file. The images are small (Alpine-based), so the whole stack, including the monitoring containers, comfortably fits on a single low-cost cloud instance.

2.2 Economic Feasibility

There are no licensing costs anywhere in the stack – Docker, Jenkins, SonarQube’s Community Edition, Nginx, Prometheus, Loki and Grafana are all free and open source, and Let’s Encrypt issues the TLS certificate for the domain at no cost. The only recurring cost is the small VPS the project is hosted on at <https://learning.run.place>, which is well within what a student project can sustain.

2.3 Operational Feasibility

Because the whole stack is defined in a handful of Compose files and a Jenkinsfile, standing it up, tearing it down, or moving it to a different server is a small number of documented commands rather than a manual, error-prone process. The health-check dashboard and, once added, the Grafana dashboards mean that a problem with any one of the fourteen services is visible immediately rather than discovered from a user complaint.

2.4 Need and Significance

College coursework covers data structures, databases and individual application development well, but rarely the question this project is built around: once an application is written, how does an

organisation actually run many pieces of it reliably, ship changes to it safely, and know when something has gone wrong? That gap is exactly what this training placement, and this project, are meant to close – it takes the CI/CD, containerization, reverse-proxy and observability practices seen at AppSmartz and turns them into a working system I can point to and explain, rather than a slide describing them.

3 Methodology / Planning of Work

3.1 Development Stages

The work is organised as a sequence of stages, most already completed and the remainder planned for the rest of the training period:

1. **Requirement analysis and service decomposition** – identify the independent business capabilities (users, products, orders, cart, inventory, payments, notifications, reviews, auth, shipping, search, analytics) and give each its own service boundary and data.
2. **Service development** – implement each service as a small Express application exposing a REST API and a `/health` endpoint used later by both the dashboard and Prometheus.
3. **API Gateway** – build a single Express-based gateway that proxies `/api/*` routes to the correct service, so the outside world only ever talks to one address.
4. **Containerization** – write a `Dockerfile` per service and two Compose files: one that builds every service from source for local development, one that pulls tagged images for deployment.
5. **API documentation** – write an OpenAPI 3 specification covering every route and serve it as an interactive Swagger UI from the gateway at `/api-docs`.
6. **CI/CD pipeline** – a Jenkinsfile that checks the repository out, runs a **SonarQube** static-analysis quality gate on the codebase, builds and tags a Docker image per service, pushes the images to Docker Hub, and deploys the production compose file to the server over SSH.
7. **Public hosting with TLS** – point the domain <https://learning.run.place> at the deployment server and put an Nginx reverse proxy in front of the frontend/gateway, terminating HTTPS with a Let's Encrypt certificate and redirecting plain HTTP to HTTPS.
8. **Observability stack** – add Prometheus to scrape metrics from the gateway and every service, Promtail to ship container logs into Loki, and Grafana dashboards that bring request rate, latency, error rate and searchable logs together in one place.
9. **Testing** – exercise every endpoint through the Swagger UI and the dashboard's request

console, and confirm the health grid, the Grafana dashboards and the HTTPS certificate all behave correctly under normal and failure conditions (e.g. a service stopped deliberately).

10. **Documentation and submission** – record the final architecture and write up this synopsis and the subsequent training report.

3.2 System Architecture Diagram

The overall request path, the CI/CD path and the observability path that result from these stages are shown together in Fig. 3.1.

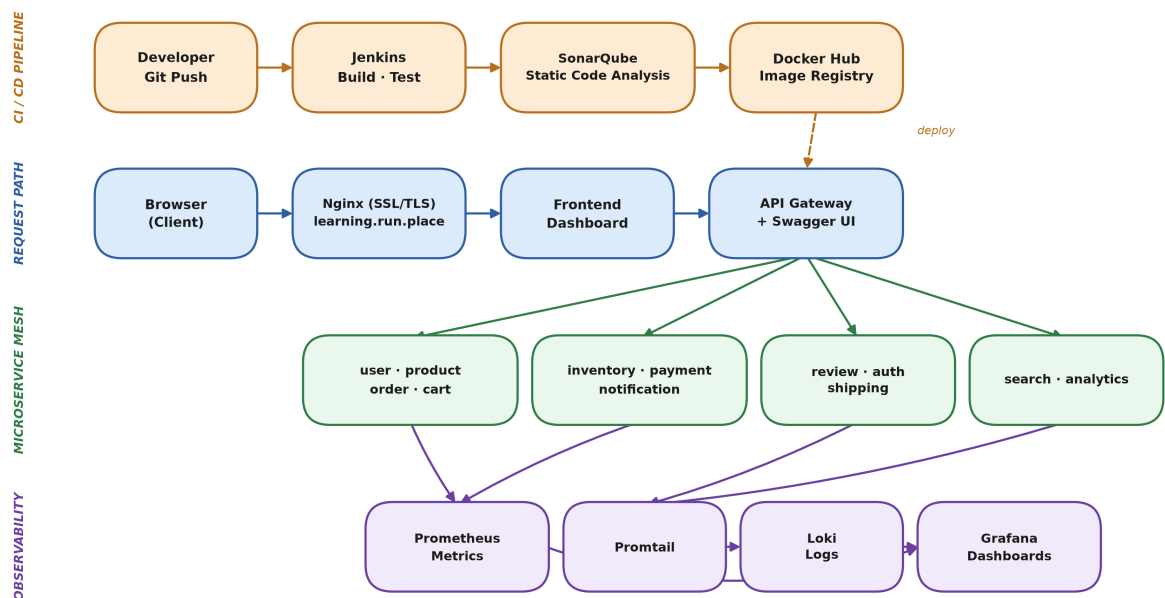


Fig. 3.1: CI/CD pipeline with the SonarQube quality gate (top), request path (middle), the microservice mesh, and the observability path (bottom).

4 Facilities Required for Proposed Work

4.1 Software

- Node.js and Express.js – backend services and API gateway
- Docker and Docker Compose – containerization and local orchestration
- Jenkins – CI/CD pipeline (build, push, deploy)
- SonarQube – static code analysis and quality gate
- Nginx and Certbot (Let's Encrypt) – reverse proxy and TLS for <https://learning.run.place>
- Prometheus – metrics collection
- Grafana Loki and Promtail – centralised log aggregation
- Grafana – dashboards and visualisation
- Swagger UI / OpenAPI 3 – API documentation
- Git and GitHub – version control
- Visual Studio Code – development environment

4.2 Hardware

- Development laptop (minimum 8 GB RAM) for local Docker Compose builds
- A cloud VPS (Ubuntu, 2 vCPU / 4 GB RAM or better) hosting the domain <https://learning.run.place>
- Stable internet connectivity for pushing/pulling container images and remote deployment

Bibliography

1. Docker Inc., *Docker Documentation*, <https://docs.docker.com>
2. Docker Inc., *Docker Compose Documentation*, <https://docs.docker.com/compose/>
3. Jenkins Project, *Jenkins User Documentation*, <https://www.jenkins.io/doc/>
4. SonarSource, *SonarQube Documentation*, <https://docs.sonarsource.com/sonarqube/latest/>
5. Nginx Inc., *Nginx Documentation*, <https://nginx.org/en/docs/>
6. Electronic Frontier Foundation, *Certbot / Let's Encrypt Documentation*, <https://certbot.eff.org>
7. Prometheus Authors, *Prometheus Documentation*, <https://prometheus.io/docs/>
8. Grafana Labs, *Grafana Documentation*, <https://grafana.com/docs/grafana/latest/>
9. Grafana Labs, *Loki Documentation*, <https://grafana.com/docs/loki/latest/>
10. OpenAPI Initiative, *OpenAPI Specification v3.0*, <https://spec.openapis.org/oas/v3.0.3>
11. SmartBear Software, *Swagger UI Documentation*, <https://swagger.io/tools/swagger-ui/>
12. OpenJS Foundation, *Express.js Documentation*, <https://expressjs.com>
13. Sam Newman, *Building Microservices*, 2nd Edition, O'Reilly Media, 2021.